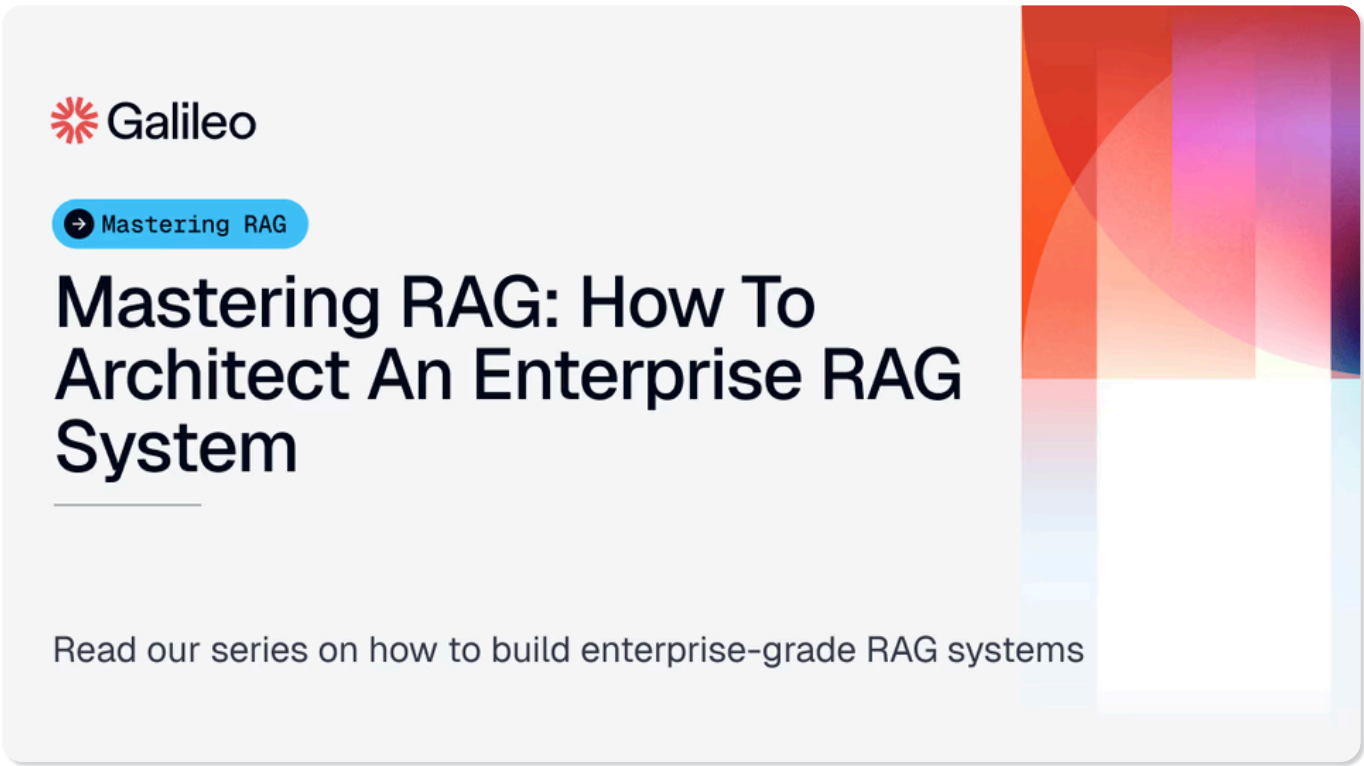
Galileo is now part of Cisco

Jan 23, 2024

Mastering RAG: How To Architect An Enterprise RAG System



Pratik Bhavsar
Evals & Leaderboards @ Galileo Labs



Explore our research-backed evaluation metric for RAG – read our paper on Chainpoll.

Welcome back to our Mastering RAG series! Let's roll up our sleeves and delve into the intricate

CONTENTS

Challenges in Building RAG Systems
7 Failure Points of RAG Systems
How to Build an Enterprise RAG System
Conclusion

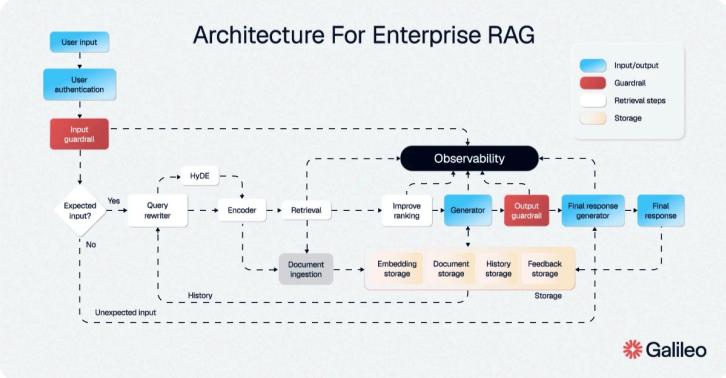
world of building an enterprise RAG system.

Subscribe

SHARE

While the internet is awash with articles on simple RAG systems, the journey to architecting a robust enterprise-level solution is often a mystery.

But this blog isn't just a theoretical journey; it's a practical guide to help you take action! From the importance of guardrails in ensuring security to the impact of query rewriting on the user experience, we're here to provide actionable insights and real-world examples. Whether you're a seasoned developer or a technical leader steering the ship, buckle up for a deep dive into the intricate world of cutting-edge enterprise RAG!



Before touching on RAG architecture, I want to share a recent study on common failure points when building RAG systems. Researchers analyzed three case studies across unique domains and found seven common RAG failure points.

Challenges in Building RAG Systems

Case Study	Domain	Doc Types	Dataset Size	RAG Stages	Sample Questions
Cognitive Reviewer*	Research	PDFs	(Any size)	Chunker, Rewriter, Retriever, Reader	What are the key points covered in this paper?
AI Tutor*	Education	Videos, HTML, PDF	38	Chunker, Rewriter, Retriever, Reader	What were the topics covered in week 6?
BioASQ	Biomedical	Scientific PDFs	4017	Chunker, Retriever, Reader	Define pseudotumor cerebri. How is it treated?

Table 1: A summary of the RAG case studies presented in this paper. Case studies marked with a * are running systems currently in use.

Cognitive reviewer

Cognitive reviewer is a RAG system designed to assist researchers in analyzing scientific documents. Researchers define a research question or objective and upload a collection of related research papers. The system then ranks all documents based on the stated objective for manual review by the researcher. Additionally, researchers can pose questions directly against the entire document set.

AI tutor

AI Tutor, another RAG system, enables students to ask questions about a unit and receive answers sourced from learning content. Students can verify answers by accessing a sources list. Integrated into Deakin's learning management system, AI Tutor indexes all content, including PDFs, videos, and text documents. The system transcribes videos using the Whisper deep learning model before chunking. The RAG pipeline incorporates a rewriter for query generalization, and a chat interface utilizes past dialogues for context in each question.

Biomedical Q&A

In the Biomedical Question and Answer case study, a RAG system was created using the BioASQ dataset, containing questions, document links, and answers. The dataset, prepared by biomedical experts, includes domain-specific question-answer pairs. The answers to questions were either yes/no, text summarisation, factoid, or list.

7 Failure Points of RAG Systems

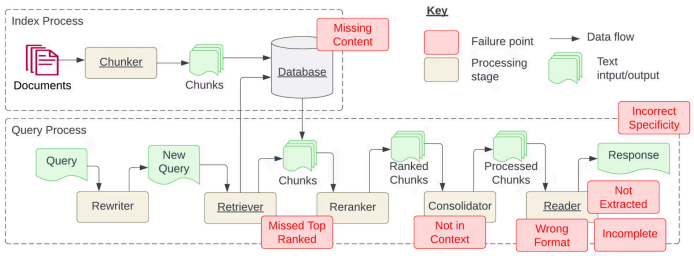


Figure 1: Indexing and Query processes required for creating a Retrieval Augmented Generation (RAG) system. The indexing process is typically done at development time and queries at runtime. Failure points identified in this study are shown in red boxes. All required stages are underlined. Figure expanded from [19].

Through these case studies, they identified seven failure points that often arise when engineering a RAG system.

Missing content (FP1)

A question is posed that cannot be answered with the available documents. In the ideal scenario, the RAG system responds with a message like "Sorry, I don't know." However, for questions related to content without clear answers, the system might be misled into providing a response.

Missed the top ranked documents (FP2)

The answer to a question is present in the document, but did not rank highly enough to be included in the results returned to the user. While all documents are theoretically ranked and utilized in subsequent steps, in practice only the top K documents are returned, with K being a value selected based on performance.

Not in context (FP3)

Documents containing the answer are retrieved from the database but fail to make it into the context for generating a response. This occurs when a substantial number of documents are returned, leading to a consolidation process where the relevant answer retrieval is hindered.

Not extracted (FP4)

The answer is present in the context, but the model fails to extract the correct information. This

typically happens when there is excessive noise or conflicting information in the context.

Wrong format (FP5)

The question involves extracting information in a specific format, such as a table or list, and the model disregards the instruction.

Incorrect specificity (FP6)

The response includes the answer but lacks the required specificity or is overly specific, failing to address the user's needs. This occurs when RAG system designers have a predetermined outcome for a given question, such as teachers seeking educational content. In such cases, specific educational content should be provided along with answers. Incorrect specificity also arises when users are unsure how to phrase a question and are too general.

Incomplete (FP7)

Incomplete answers are accurate but lack some information, even though that information was present in the context and available for extraction. For instance, a question like "What are the key points covered in documents A, B, and C?" would be better approached by asking these questions separately.

FP	Lesson	Description	Case Studies
FP4	Larger context get better results (Context refers to a particular setting or situation in which the content occurs)	A larger context enabled more accurate responses (8K vs 4K). Contrary to prior work with GPT-3.5 [13]	AI Tutor
FP1	Semantic caching drives cost and latency down	RAG systems struggle with concurrent users due to rate limits and the cost of LLMs. Prepopulate the semantic cache with frequently asked questions [1].	AI Tutor
FP5-7	Jailbreaks bypass the RAG system and hit the safety training.	Research suggests fine-tuning LLMs reverses safety training [11], test all fine-tuned LLMs for RAG system.	AI Tutor
FP2, FP4	Adding meta-data improves retrieval.	Adding the file name and chunk number into the retrieved context helped the reader extract the required information. Useful for chat dialogue.	AI Tutor
FP2, FP4-7	Open source embedding models perform better for small text.	Opensource sentence embedding models performed as well as closed source alternatives on small text.	BioASQ, AI Tutor
FP2-7	RAG systems require continuous calibration.	RAG systems receive unknown input at runtime requiring constant monitoring.	AI Tutor, BioASQ
FP1, FP2	Implement a RAG pipeline for configuration.	A RAG system requires calibrating chunk size, embedding strategy, chunking strategy, retrieval strategy, consolidation strategy, context size, and prompts.	Cognitive Reviewer, AI Tutor, BioASQ
FP2, FP4	RAG pipelines created by assembling bespoke solutions are suboptima.	End-to-end training enhances domain adaptation in RAG systems [18].	BioASQ, AI Tutor
FP2-7	Testing performance characteristics are only possible at runtime.	Offline evaluation techniques such as G-Evals [14] look promising but are premised on having access to labelled question and answer pairs.	Cognitive Reviewer, AI Tutor

Table 2: The lessons learned from the three case studies with key takeaways for future RAG implementations

The table below contains the lessons they learned from solving each problem. **We will keep these**

lessons in mind while architecting our enterprise RAG system.

How to Build an Enterprise RAG System

Now that we've established common problems faced when designing a RAG system, let's go through the design needs and roles of each component, along with best practices for building them. The RAG system architecture diagram above offers context on where and how each component is used.

User authentication

Where it all starts – the first component in our system! Before the user can even start interacting with the chatbot, we need to authenticate the user for various reasons. Authentication helps with security and personalization, which is a must for enterprise systems.

Access Control

Authentication ensures that only authorized users gain access to the system. It helps control who can interact with the system and which actions they are allowed to perform.

Data Security

Protecting sensitive data is paramount. User authentication prevents unauthorized individuals from accessing confidential information, preventing data breaches and unauthorized data manipulation.

User Privacy

Authentication helps maintain user privacy by ensuring that only the intended user can access

their personal information and account details. This is crucial for building trust with users.

Legal Compliance

Many jurisdictions and industries have regulations and laws that require organizations to implement proper user authentication to protect user data and privacy. Adhering to these regulations helps avoid legal issues and potential penalties.

Accountability

Authentication ensures accountability by tying actions within the system to specific user accounts. This is essential for auditing and tracking user activities, helping to identify and address any security incidents or suspicious behavior.

Personalization and Customization

Authentication allows systems to recognize individual users, enabling personalization and customization of user experiences. This can include tailored content, preferences, and settings.

Services like AWS Cognito or Firebase Authentication can help you easily add user sign-up and authentication to mobile and web apps.

Input guardrail

It's essential to prevent user inputs that can be harmful or contain private information. Recent studies have shown it's easy to jailbreak LLMs. Here's where input guardrails come in. Let's have a look at different scenarios for which we need guardrails.

Anonymization

Input guardrails can anonymize or redact personally identifiable information (PII) such as names, addresses, or contact details. This helps to

protect privacy and prevent malicious attempts to disclose sensitive information.

Restrict substrings

Prohibiting certain substrings or patterns that could be exploited for SQL injection, cross-site scripting (XSS), or other injection attacks prevents security vulnerabilities or unwanted behaviors.

Restrict topics

In order to restrict discussions or inputs related to specific topics that may be inappropriate, offensive, or violate community guidelines, it's important to filter out content that involves hate speech, discrimination, or explicit material.

Restrict code

It's essential to prevent the injection of executable code that could compromise system security or lead to code injection attacks.

Restrict language

Verify that text inputs are in the correct language or script, preventing potential misinterpretations or errors in processing.

Detect prompt injection

Mitigate attempts to inject misleading or harmful prompts that may manipulate the system or influence the behavior of LLMs in unintended ways.

Limit tokens

Enforcing a maximum token or character limit for user inputs helps avoid resource exhaustion and prevents denial-of-service (DoS) attacks.

Detect toxicity

Implement toxicity filters to identify and block inputs that contain harmful or abusive language.

To safeguard your RAG system against these scenarios, you can leverage Llama Guard by Meta. You can host it yourself or use a managed service like Sagemaker. However, do not expect it to be perfect at detecting toxicity.



Query rewriter

Once the query passes the input guardrail, we send it to the query rewriter. Sometimes, user queries can be vague or require context to understand the user's intention better. Query rewriting is a technique that helps with this. It involves transforming user queries to enhance clarity, precision, and relevance. Let's go through some of the most popular techniques.

Rewrite based on history

In this method, the system leverages the user's query history to understand the context of the conversation and enhance subsequent queries. Let’s use an example of a credit card inquiry.

Query History:

"How many credit cards do you have?"

"Are there any yearly fees for platinum and gold credit cards?"

"Compare features of both."

We must identify the context evolution based on the user's query history, discern the user's intent and relationship between queries, and generate a query that aligns with the evolving context.

Rewritten Query: "Compare features of platinum and gold credit cards."

Create subqueries

Complex queries can be difficult to answer due to retrieval issues. To simplify the task, queries are broken down into more specific subqueries. This helps to retrieve the right context needed for generating the answer. LlamaIndex refers to this as a sub question query engine.

Given the query "Compare features of platinum and gold credit card," the system generates subqueries for each card that focus on individual entities mentioned in the original query.

Rewritten Subqueries:

1. "What are the features of platinum credit cards?"
2. "What are the features of gold credit cards?"

Create similar queries

To increase the chances of retrieval of the right document, we generate similar queries based on user input. This is to overcome the limitations of the retrieval in semantic or lexical matching.

If the user asks about credit card features, the system generates related queries. Use synonyms, related terms, or domain-specific knowledge to create queries that align with the user's intent.

Generated Similar Query:

“I want to know about platinum credit cards” → "Tell me about the benefits of platinum credit cards."

Encoder

Once we have the original and rewritten queries, we encode them into vectors (a list of numbers) for retrieval. ***Choosing an encoder is probably the most important decision in building your RAG system.*** Let's explore why and the factors to consider when choosing your text encoder.

Leveraging MTEB benchmarks

For a comprehensive assessment of encoder capabilities, the go-to source is the Massive Text Embedding Benchmark (MTEB). This benchmark allows for a nuanced selection of encoders based on vector dimension, average retrieval performance, and model size. The landscape has undergone significant evolution since early 2024.

OpenAI's text-embedding-3-large and text-embedding-3-small models now offer dimension flexibility, allowing you to reduce embedding dimensions while maintaining performance. These models support dimensions from 256 to 3072, enabling a trade-off between performance and storage costs.

Cohere's Embed v3 has introduced multilingual support for over 100 languages with a single model, eliminating the need for separate encoders per language. Voyage AI's voyage-large-2-instruct and voyage-code-2 have shown impressive performance on domain-specific tasks, particularly for code and technical documentation.

For open-source alternatives, the BGE-M3 model now supports multiple retrieval modes (dense, sparse, and multi-vector) within a single model, whereas models like E5-mistral-7b-instruct utilize instruction-tuning for improved query understanding. The Jina embeddings v2 family offers models optimized for different context lengths, with their 8K variant handling long documents effectively.

When selecting models today, consider not just MTEB scores, but also inference latency requirements, multilingual needs, maximum context length, and whether you need specialized domain performance. Many teams are finding success with ensemble approaches, using different models for different document types within the same system.

It's essential to note that MTEB results provide a general overview and may not accurately indicate how well these embeddings will perform within the specific context of your domain. Therefore, **it is imperative to perform a thorough evaluation on your dataset before making a final selection**, emphasizing the significance of custom evaluation methodologies.

Custom evaluation

Encoders may not consistently deliver optimal performance, especially when handling sensitive information. Custom evaluation methods become crucial in such scenarios. Here are three approaches to performing custom evaluations.

Evaluation by annotation

Generate a dedicated dataset and set up annotations to obtain gold labels. After annotation, leverage retrieval metrics like Mean Reciprocal Rank (MRR) and Normalized Discounted Cumulative Gain (NDCG) to assess the performance of different encoders quantitatively.

Evaluation by model

Follow a data generation process similar to the annotation approach, but use an LLM or a cross-encoder as the evaluator. This allows the establishment of a relative ranking among all

encoders. Subsequently, manual assessment of the top three encoders can yield precise performance metrics.

Evaluation by clustering

Employ diverse clustering techniques and analyze the coverage (quantity of data clustered) at distinct Silhouette scores, indicating vector similarity within clusters. Experiment with algorithms like HDBSCAN, adjusting their parameters for optimal performance selection. This clustering-based evaluation provides valuable insights into the distribution and grouping of data points, aiding in selecting encoders that align with specific metrics.

How to Select a Text Encoder

When choosing your encoder, you'll need to decide between a private encoder or a public encoder. You might be tempted to use a private encoder due to their ease-of-use, but there are specific tradeoffs that require consideration between the two options. It's an important decision that will decide the performance and latency of your system.

Querying cost

Ensuring a smooth user experience in semantic search relies on the high availability of the embedding API service. OpenAI and similar providers offer reliable APIs, eliminating the need for hosting management. Opting for an open-source model, however, requires engineering efforts based on model size and latency needs. Smaller models (up to 110M parameters) can be hosted with a CPU instance, while larger models may demand GPU serving to meet latency requirements.

Indexing cost

Setting up semantic search involves indexing documents, incurring a non-trivial cost. As indexing and querying share the same encoder, the indexing cost hinges on the chosen encoder service. To facilitate service resets or reindexing onto an alternative vector database, it's advisable to store embeddings separately. Neglecting this step would necessitate recalculating identical embeddings.

Storage Cost

For applications indexing millions of vectors, Vector DB storage cost is a crucial consideration. Storage cost scales linearly with dimension, and OpenAI's embeddings in 1526 dimensions incur the maximum storage cost. To estimate storage cost, calculate average units (phrase or sentence) per doc and extrapolate.

Language Support

In order to support your non-English language either use a multi-lingual encoder or use a translation system along with an English encoder.

Search latency

The latency of semantic search grows linearly with the dimension of the embeddings. Opting for lower-dimensional embeddings is preferable to minimize latency.

Privacy

Stringent data privacy requirements in sensitive domains like finance and healthcare may render services like OpenAI less viable.



Document ingestion

The Document ingestion system manages the processing and persistence of data. During the indexing process each document is split into smaller chunks that are converted into an embedding using an embedding model. The original chunk and the embedding are then indexed in a database. Let's look at the components of the document ingestion system.

Document parser

The document parser takes a central role in actively extracting structured information from diverse document formats, with a particular focus on format handling. This includes, but is not limited to, parsing PDFs that may contain images and tables.

Document formats

The document parser must demonstrate proficiency in handling a variety of document formats, such as PDF, Word, Excel, and others, ensuring adaptability in document processing. This involves identifying and managing embedded content, such as hyperlinks, multimedia elements, or annotations, to provide a comprehensive representation of the document.

Table recognition

Recognizing and extracting data from tables within documents is imperative for maintaining the structure of information, especially in reports or research papers. The extraction of metadata related to tables, including headers, row and column information, enhances the comprehension of the document's organizational structure. Models such as Table Transformer can be useful for the task.

Image recognition

OCR is applied to images within documents to actively recognize and extract text, making it accessible for indexing and subsequent retrieval.

Metadata extraction

Metadata refers to additional information about the document that is not part of its main content. It includes details such as author, creation date, document type, keywords, etc. Metadata provides valuable context and helps in organizing documents and to improve the relevance of search results by considering metadata attributes. The metadata can be extracted with an NLP/OCR pipeline and indexed with the docs as special fields.

Chunker

How you decide to tokenize (break) longform text can decide the quality of your embeddings and the performance of your search system. If chunks are too small, certain questions cannot be answered; if the chunks are too long, then the answers include generated noise. You can exploit summarisation techniques to reduce noise, text size, encoding cost and storage cost.

Chunking is an important yet underrated topic. It can require domain expertise similar to feature

engineering. For example, chunking for python codebases might be done with prefixes like def/class. Read our blog for a deeper dive into chunking.



Indexer

The indexer is – you guessed it – responsible for creating an index of the documents, which serves as a structured data structure (say that 3 times fast…). The indexer facilitates efficient search and retrieval operations. **Efficient indexing is crucial for quick and accurate document retrieval.** It involves mapping the chunks or tokens to their corresponding locations in the document collection. The indexer performs vital tasks in document retrieval, including creating an index and adding, updating, or deleting documents.

The indexer, being a critical component of a RAG system, faces various challenges and issues that can impact the overall efficiency and performance of the system.

Scalability issues

As the volume of documents grows, maintaining efficient and fast indexing becomes challenging. Scalability issues may arise when the system struggles to handle an increasing number of documents, leading to slower indexing and retrieval times.

Real-time index updates

Keeping the index up-to-date in real-time can be challenging, especially in systems where documents are frequently added, updated, or deleted. Ensuring that live APIs and real-time indexing mechanisms operate seamlessly without compromising system performance is a persistent challenge.

Consistency and atomicity

Achieving consistency and atomicity in the face of concurrent document updates or modifications can be complex. Ensuring that updates to the index maintain data integrity, even in the presence of simultaneous changes, requires careful design and implementation.

Optimizing storage space

Indexing large volumes of documents may lead to considerable storage requirements. Optimizing storage space while ensuring that the index remains accessible and responsive is an ongoing challenge, especially in scenarios where storage costs are a concern.

Security and access control

Implementing proper security measures and access controls to prevent unauthorized modifications to the index is crucial. Ensuring that only authorized users or processes can perform CRUD operations helps protect the integrity of the document repository.

Monitoring and maintenance

Regularly monitoring the health and performance of the indexer is essential. Detecting issues, such as indexing failures, resource bottlenecks, or outdated indexes, requires robust monitoring and maintenance procedures to ensure the system operates smoothly over time.

These are some difficult but well known software engineering challenges which can be tackled by following good software design practices.

Data storage

Since we are dealing with a variety of data we need dedicated storage for each of them. It's critical to understand the different considerations for every storage type and specific use cases of each.

Embeddings

Database type: SQL/NoSQL

Storing document embeddings separately allows for swift reindexing without recalculating embeddings for the entire document corpus. Additionally, embedding storage acts as a backup, ensuring the preservation of critical information even in the event of system failures or updates.

Documents

Database type: NoSQL

Document storage in its raw format is essential for persistent storage. This raw format serves as the foundation for various processing stages, such as indexing, parsing, and retrieval. It also provides flexibility for future system enhancements, as the original documents remain intact and can be reprocessed as needed.

Chat history

Database type: NoSQL

The storage of chat history is imperative for supporting the conversational aspect of the RAG system. Chat history storage allows the system to recall previous user queries, responses, and preferences, enabling it to adapt and tailor future interactions based on the user's unique context. This historical data is a valuable resource for

improving the ML system by leveraging it for research.

User feedback

Database type: NoSQL/SQL

User feedback is systematically collected through various interaction mechanisms within the RAG application. In most LLM systems, users can provide feedback using thumbs-up/thumbs-down, star ratings and text feedback. This array of user insights serves as a valuable repository, encapsulating user experiences and perceptions, forming the basis for ongoing system enhancements.

Vector database

The vector database powering the semantic search is a crucial retrieval component of RAG. However, selecting this component appropriately is vital to avoid potential issues. Several vector database factors need to be considered in the selection process. Let's go over some of them.

Recall vs. Latency

Optimizing for recall (percentage of relevant results) versus latency (time to return results) is a trade-off in vector databases. Different indexes like Flat, HNSW (Hierarchical Navigable Small World), PQ (Product quantization), ANNOY, and DiskANN make varying trade-offs between speed and recall. Conduct benchmark studies on your data and queries to make an informed decision.

Cost

Cloud-native databases with managed solutions typically bill based on data storage and query volume. This model is suitable for organizations with substantial data, avoiding infrastructure costs. Key considerations include evaluating dataset growth, the team's capability, data sensitivity, and

understanding the cost implications of managed cloud solutions.

On the other side, self-hosting provides organizations with more control over their infrastructure and potentially lower costs. However, it comes with the responsibility of managing and maintaining the infrastructure, including considerations for scalability, security, and updates.

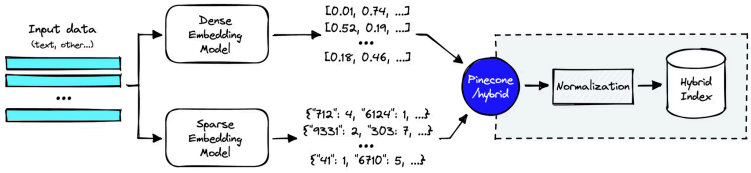
Insertion speed vs. Query speed

Balancing insertion speed and query speed is vital. Look for vendors that can handle streaming use cases with high insertion speed requirements. However, for most organizations, prioritizing querying speed is more relevant. Evaluate the vector insertion speed query latency at peak loads to make an informed decision.

In-memory vs. On-disk index storage

Choosing between in-memory and on-disk storage involves speed and cost trade-offs. While in-memory storage offers high speed, some use cases require storing vectors larger than memory. Techniques like memory-mapped files allow scaling vector storage without compromising search speed. New indexes like Vamana in DiskANN promise efficient out-of-memory indexing.

Full-Text search vs. Vector Hybrid search



Source: https://www.pinecone.io/learn/hybrid-search-intro/

Vector search alone may not be optimal for enterprise-level applications. On the other hand, hybrid search, which integrates both dense and

sparse methodologies, requires additional effort. Implementing a dense vector index, a sparse inverted index, and a reranking step is typical. The balance between dense and sparse elements is adjustable through a parameter known as alpha in Pinecone, Weaviate & Elasticsearch.

Filtering

Real-world search queries often involve filtering on metadata attributes. Pre-filtered search, although seemingly natural, can lead to missing relevant results. Post-filtered search may have issues if the filtered attribute is a small fraction of the dataset. Custom-filtered search, like Weaviate, combines pre-filtering with effective semantic search using inverted index shards alongside HNSW index shards.

Techniques for improving retrieval

Recent research has shown that LLMs can be easily distracted by irrelevant context and having a lot of context (topK retrieved docs) can lead to missing out of certain context due to the attention patterns of LLMs. Therefore it is crucial to improve retrieval with relevant and diverse documents. Let's look at some of the proven techniques for improving retrieval.

Hypothetical document embeddings (HyDE)

We can use the HyDE technique to tackle the problem of poor retrieval performance, especially when dealing with short or mismatched queries that can make finding information difficult. HyDE takes a unique approach by using hypothetical documents created by models like GPT. These hypothetical documents capture important patterns but might have made-up or incorrect details. A smart text encoder then turns this hypothetical document into a vector embedding.

This embedding helps find similar real documents in the collection better than embedding of query.

Experiments show that HyDE works better than other advanced methods, making it a useful tool to boost the performance of RAG systems.

Query routing

Query routing proves advantageous when dealing with multiple indexes, directing queries to the most relevant index for efficient retrieval. This approach streamlines the search process by ensuring that each query is directed to the appropriate index, optimizing the accuracy and speed of information retrieval.

In the context of enterprise search, where data is indexed from diverse sources such as technical documents, product documentation, tasks, and code repositories, query routing becomes a powerful tool. For instance, if a user is searching for information related to a specific product feature, the query can be intelligently routed to the index containing product documentation, enhancing the precision of search results.

Reranker

When retrieval from the encoder falls short of delivering optimal quality, a reranker is used to enhance the document ranking. Utilizing open-source encoder-only transformers like BGE-large in a cross-encoder setup has become a common practice. Recent decoder-only approaches, such as RankVicuna, RankGPT, and RankZephyr have further boosted reranker performance.

Introducing a reranker has benefits, reducing LLM hallucinations in responses and improving the system's out-of-domain generalization. However, it comes with drawbacks. Sophisticated rerankers may increase latency due to computational overhead, impacting real-time applications.

Additionally, deploying advanced rerankers can be resource-intensive, demanding careful consideration of the balance between performance gains and resource utilization.

Maximal Marginal Relevance (MMR)

MMR is a method designed to enhance the diversity of retrieved items in response to a query, avoiding redundancy. Rather than focusing solely on retrieving the most relevant items, MMR achieves a balance between relevance and diversity. It's like introducing a friend to people at a party. Initially, it identifies the most matching person based on the friend's preferences. Then, it seeks someone slightly different. This process continues until the desired number of introductions is achieved. MMR ensures a more diverse and relevant set of items is presented, minimizing redundancy.

Autocut

The autocut feature from Weaviate, is designed to limit the number of search results returned by detecting groups of objects with close scores. It works by analyzing the scores of the search results and identifying significant jumps in these values, which can indicate a transition from highly relevant to less relevant results.

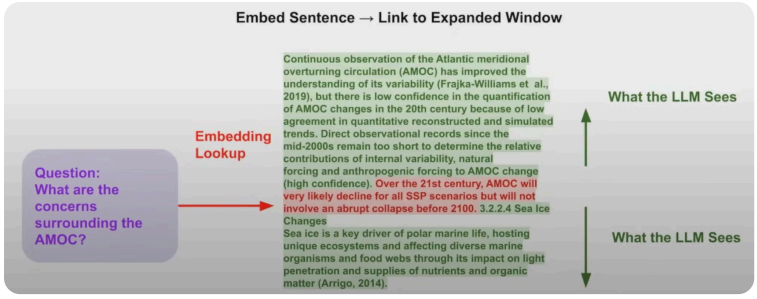
For example, consider a search that returns objects with these distance values:

[0.1899, 0.1901, 0.191, 0.21, 0.215, 0.23].

Autocut returns the following:

- autocut: 1: [0.1899, 0.1901, 0.191]
- autocut: 2: [0.1899, 0.1901, 0.191, 0.21, 0.215]
- autocut: 3: [0.1899, 0.1901, 0.191, 0.21, 0.215, 0.23]

Recursive retrieval



Source: https://youtu.be/TRjq7t2Ms5I?si=D0z5sHKW4SMqMgSG&t=742

Recursive retrieval, aka the small-to-big retrieval technique, embeds smaller chunks for retrieval while returning larger parent context for the language model's synthesis. Smaller text chunks contribute to more accurate retrieval, while larger chunks provide richer contextual information for the language model. This sequential process optimizes the accuracy of retrieval by initially focusing on smaller, more information-dense units, which are then efficiently linked to their broader contextual parent chunks for synthesis.

Sentence window retrieval

The retrieval process fetches a single sentence and returns a window of text around that particular sentence. Sentence window retrieval ensures that the information retrieved is not only accurate but also contextually relevant, offering comprehensive information around the main sentence.

Generator

Now that we've discussed all retrieval components let's talk about the generator. It requires careful considerations and trade-offs mainly between self-hosted inference deployment and private API services. This is a large topic in itself, and I will touch on it briefly to avoid overwhelming you.

API considerations

When evaluating an API server for LLMs, it is crucial to prioritize features that ensure seamless

integration and robust performance. A well-designed API should function as a simple launcher for popular LLMs, while also addressing key considerations such as production readiness, security, and hallucination detection. Notably, the TGI server from HuggingFace exemplifies a comprehensive set of features that embody these principles. Let's understand some of the most popular features needed in a LLM server.

Performance

An efficient API must prioritize performance to cater to diverse user needs. Tensor parallelism stands out as a feature that facilitates faster inference on multiple GPUs, enhancing the overall processing speed. Additionally, continuous batching of incoming requests ensures an increased total throughput, contributing to a more responsive and scalable system. The inclusion of quantization, specifically with bitsandbytes and GPT-Q, further optimizes the API for enhanced efficiency across various use cases. The ability to utilize optimized transformers code ensures seamless inference on the most popular architectures.

Generation quality enhancers

To elevate the quality of generation, the API should incorporate features that can transform the output. The logits processor, encompassing temperature scaling, top-p, top-k, and repetition penalty, allows users to customize the output according to their preferences. Moreover, a stop sequences provides control over the generation, enabling users to manage and refine the content generation process. Log probabilities, crucial for hallucination detection, serve as an additional layer of refinement, ensuring that the generated output aligns with the intended context and avoids misleading information.

Security

The security of an API is paramount, particularly when dealing with LLMs and enterprise use cases. Safetensors weight loading is an essential feature, contributing to the secure deployment of models by preventing unauthorized tampering with model parameters. Furthermore, the inclusion of watermarking adds an extra layer of security, enabling traceability and accountability in the usage of LLMs.

User experience

In the realm of user experience, token streaming emerges as a critical feature for seamless interaction. Utilizing Server-Sent Events (SSE) for token streaming enhances the real-time responsiveness of the API, providing users with a smoother and more interactive experience. This ensures that users can receive generated content incrementally, improving the overall engagement and usability of the LLM.

Self-hosted inference

Self-hosted inference involves deploying LLMs on servers provided by cloud service providers like AWS, GCP, or Azure. The choice of servers, such as TGI, Ray, or FastAPI, is a critical decision that directly impacts the system's performance and cost. Considerations include computational efficiency, ease of deployment, and compatibility with the selected LLM.

Measuring LLM inference performance is crucial, and leaderboards like Anyscale's LLMPerf Leaderboard are invaluable. It ranks inference providers based on key performance metrics, including time to first token (TTFT), inter-token latency (ITL), and success rate. Load tests and

correctness tests are vital for evaluating different characteristics of hosted models.

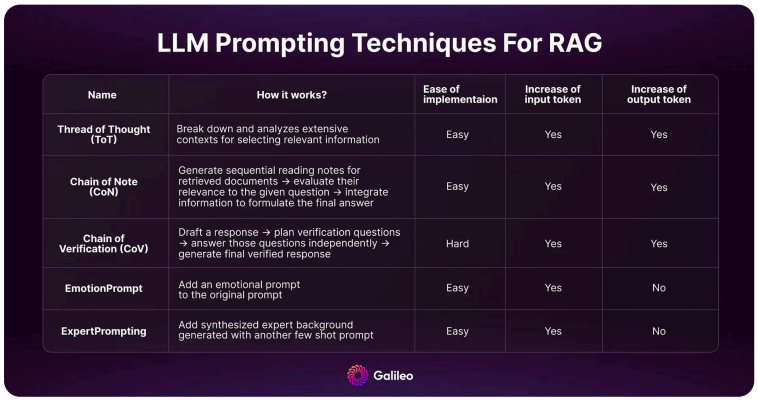
In new approaches, Predibase's LoRAX introduces an innovative way to serve fine-tuned LLMs efficiently. It addresses the challenge of serving multiple fine-tuned models using shared GPU resources.

Private API services

LLM API services by companies like OpenAI, Fireworks, Anyscale, Replicate, Mistral, Perplexity and Together, present alternative deployment approaches. It's essential to understand their features, pricing models, and LLM performance metrics. For instance, OpenAI's token-based pricing, with distinctions between input and output tokens, can significantly impact the overall cost of using the API. When comparing the cost of private API services versus self-hosted LLMs, you must consider factors such as GPU costs, utilization, and scalability issues. For some, rate limits can be a limiting factor (ha!).

Prompting techniques for improving RAG

Several prompting techniques exist for improving RAG outputs. In part 2 of our Mastering RAG series we did a deep dive into the top 5 most effective ones. Many of these new techniques surpass the performance of CoT (Chain-of-Thought). You can also combine them to minimize hallucinations.



Output guardrail

The output guardrail functions similarly to its input counterpart but is specifically tailored to detect issues in the generated output. It focuses on identifying hallucinations, competitor mentions, and potential brand damage as part of RAG evaluation. The goal is to prevent generating inaccurate or ethically questionable information that may not align with the brand's values. By actively monitoring and analyzing the output, this guardrail ensures that the generated content remains factually accurate, ethically sound, and consistent with the brand's guidelines.

Here is an example of a response that can damage an enterprise brand but would be blocked by an appropriate output guardrail:



User feedback

Once an output is generated and served, it is helpful to get both positive or negative feedback from users. User feedback can be very helpful for improving the flywheel of the RAG system, which is a continuous journey rather than a one-time endeavor. This entails not only the routine execution of automated tasks like reindexing and experiment reruns but also a systematic approach to integrate user insights for substantial system enhancements.

The most impactful lever for system improvement lies in the active remediation of issues within the underlying data. RAG systems should include an iterative workflow for handling user feedback and driving continuous improvement.

User interaction and feedback collection

Users interact with the RAG application and utilize features such as 👍 / 👎 or star ratings to provide feedback. This diverse set of feedback mechanisms serves as a valuable repository of user experiences and perceptions regarding the system's performance.

Issue identification and diagnostic inspection

After collecting feedback, the team can conduct a comprehensive analysis to identify queries that may be underperforming. This involves inspecting retrieved resources and scrutinizing to discern whether underperformance stems from retrieval, generation, or the underlying data source.

Data improvement strategies

Once issues are identified, particularly those rooted in the data itself, the team can strategically devise plans to enhance data quality. This may involve rectifying incomplete information or restructuring poorly organized content.

Evaluation and testing protocols

After implementing data improvements, the system must undergo rigorous evaluation on previously underperforming queries. Insights gained from these evaluations can then be methodically integrated into the test suite, ensuring ongoing scrutiny and refinement based on real-world interactions.

By actively engaging users in this comprehensive feedback loop, the RAG system not only addresses issues identified through automated processes but also harnesses the richness of user experiences.

Observability

Building a RAG system does not end with putting the system into production. Even with robust guardrails and high-quality data for fine-tuning, models require constant monitoring once in production. Generative AI apps, in addition to standard metrics like latency and cost, need specific LLM observability to detect and correct issues such as hallucinations, out-of-domain queries, and chain failures. Now let's have a look at the pillars of LLM observability.

Prompt analysis and optimization

Identify prompt-related problems and iterate using live production data to identify and address issues like hallucinations using robust evaluation mechanisms.

Traceability in LLM applications

Capture LLM traces from common frameworks like Langchain and LlamaIndex to debug prompts and steps.

Information retrieval enhancement

Troubleshoot and evaluate RAG parameters to optimize retrieval processes critical to LLM performance.

Alerting

Get alerts if system behavior diverges from the expected, such as increased errors, high latency and hallucinations.

First and foremost, real-time monitoring is essential to observe the performance, behavior, and overall health of your applications in a production environment. Keep a vigilant eye on SLA compliance and set up alerts to promptly address any deviations. Effectively track the costs associated with running LLM applications by analyzing usage patterns and resource consumption to help you in cost optimization.

Galileo offers purpose-built LLM observability to alert and take immediate corrective action before user complaints proactively. Galileo's guardrail metrics are designed to monitor the quality and safety of your models, covering aspects like groundedness, uncertainty, factuality, tone, toxicity, PII, and more. These metrics, previously utilized during evaluation and experimentation, can now be seamlessly integrated into the monitoring phase.

Additionally, you have the flexibility to register custom metrics to tailor the monitoring process to your specific needs. Leverage insights and alerts generated from monitoring data to stay informed about potential issues, anomalies, or areas for improvement that demand attention. This comprehensive approach ensures that your LLM applications operate efficiently and securely in real-world scenarios.

Caching

For companies operating at scale, cost can become a hindrance. Caching is a great way to save money in such cases. Caching involves the storage of prompts and their corresponding responses in a database, enabling their retrieval for subsequent use. This strategic caching mechanism empowers LLM applications to expedite and economize responses with three distinct advantages.

Enhanced production inference

Caching contributes to faster and more cost-effective inference during production. Certain queries can achieve near-zero latency by leveraging cached responses, streamlining the user experience.

Accelerated development cycles

In the development phase, caching proves to be a boon as it eliminates the need to invoke the API for

identical prompts repeatedly. This results in faster and more economical development cycles.

Data storage

The existence of a comprehensive database storing all prompts simplifies the fine-tuning process for LLMs. Utilizing the stored prompt-response pairs streamlines the optimization of the model based on accumulated data.

If you are serious, you can leverage GPTCache to implement caching for exact and similar matches. It offers valuable metrics such as cache hit ratio, latency, and recall, which provide insights into the cache's performance, enabling continuous refinement to ensure optimal efficiency.

Advanced RAG

While traditional RAG systems have proven effective for many use cases, the demands of modern enterprise applications have exposed their limitations. Simple retrieve-and-generate pipelines struggle with complex reasoning tasks, fail to capture intricate relationships between entities, and are confined to text-only knowledge bases. These constraints have driven the development of advanced RAG architectures that fundamentally reimagine the traditional system. In this section, we explore three transformative approaches that are reshaping the RAG landscape in last few years.

Agentic RAG

Agentic RAG represents a paradigm shift from simple retrieve-and-generate pipelines to systems that actively reason about information needs. Unlike traditional RAG, agentic systems can dynamically decide when to search, what to search for, and how to validate retrieved information.

Key components of Agentic RAG include:

Adaptive Retrieval Planning: The system determines whether retrieval is necessary and plans multi-step retrieval strategies. For complex queries requiring information from multiple sources, the agent breaks down the task into sub-queries, retrieves relevant information iteratively, and synthesizes findings.

Self-Reflection and Verification: Agents can assess the quality of retrieved documents and their own responses. If the initial retrieval doesn't fully answer the question, the agent reformulates queries or seeks additional sources. This includes fact-checking against multiple sources and identifying potential contradictions.

Tool Integration: Beyond simple document retrieval, agentic RAG systems integrate multiple tools—calculators for numerical verification, code interpreters for technical validation, or API calls for real-time data. The agent decides which tools to use based on the query context.

Memory and Context Management: Agents maintain conversation state and can reference previous interactions, building a more coherent understanding over multi-turn conversations. They intelligently manage context windows, summarizing less relevant information while preserving critical details.

Implementation considerations include higher computational costs due to multiple LLM calls, the need for robust error handling as complexity increases, and careful prompt engineering to guide agent behavior without over-constraining flexibility.

Graph RAG

Graph RAG combines the power of knowledge graphs with vector-based retrieval to capture complex relationships between entities that traditional RAG systems might miss. By representing information as interconnected nodes

and edges, Graph RAG excels at multi-hop reasoning and relationship-aware retrieval.

The architecture typically involves:

Knowledge Graph Construction: Documents are processed to extract entities, relationships, and attributes. Natural language processing identifies named entities, relationship extraction models determine connections, and community detection algorithms identify clusters of related information. This creates a structured representation that preserves semantic relationships beyond what embeddings alone can capture.

Hybrid Retrieval Strategy: Graph RAG employs both graph traversal and vector similarity. Initial retrieval might use vector search to find relevant nodes, then graph traversal explores connected entities up to a specified depth. This approach answers questions like "What are all the downstream effects of X?" or "How does A relate to B through intermediate connections?"

Graph-Aware Summarization: Retrieved subgraphs are intelligently summarized, preserving relationship context. The system can generate explanations that explicitly reference the path through the knowledge graph, thereby providing transparency in the reasoning process.

Practical applications excel in domains with rich interconnections—financial services for risk assessment across related entities, healthcare for understanding drug interactions and comorbidities, and research for literature review and hypothesis generation. The main trade-offs include higher preprocessing costs for graph construction, increased storage requirements for maintaining graph structures, and the need for domain expertise in designing effective ontologies.

Multi-Modal RAG

Multi-modal RAG systems process and retrieve information across different modalities—text, images, audio, video, and structured data—providing richer context for generation. This approach is essential for applications dealing with diverse content types like technical documentation with diagrams, medical imaging with reports, or educational content with mixed media.

Architecture Components:

Multi-Modal Encoders: Separate encoders for each modality create unified embedding spaces. Vision transformers process images, extracting both global features and region-specific information. Audio encoders handle speech and sound, often with transcription capabilities. The challenge lies in aligning these different embedding spaces for effective cross-modal retrieval.

Cross-Modal Retrieval: Queries in one modality can retrieve relevant information in another. Text queries can find relevant images or video segments, image inputs can retrieve explanatory text or similar visual content, and audio queries can match against transcripts or related visual content. This requires careful attention to embedding alignment and similarity metrics.

Fusion Strategies: Retrieved multi-modal content must be effectively combined for generation. Early fusion concatenates features before processing, while late fusion processes modalities separately then combines results. Attention mechanisms help the model focus on relevant aspects of each modality.

Implementation Challenges: Storage costs increase significantly with multiple modalities, especially for high-resolution images or video. Processing latency varies widely across modalities, requiring careful pipeline optimization. Relevance scoring across modalities needs calibration—is a

highly relevant image more valuable than moderately relevant text?

Best practices include preprocessing strategies like image captioning and OCR for searchability, maintaining modality-specific indices with cross-references, and implementing fallback mechanisms when certain modalities aren't available.

Rollback and Experimentation Strategies

Production RAG systems require robust rollback mechanisms to handle model regressions, index corruptions, or performance degradations. Implement versioned deployments where each component (embeddings, indices, model weights) is tagged with version identifiers. Maintain at least three versions: current production, previous stable, and candidate release.

Blue-green deployment works effectively for RAG systems. Maintain two identical production environments, with traffic routed to the "blue" environment while "green" serves as standby. Updates deploy to green first, undergo validation, then traffic switches. If issues arise, immediate rollback involves simply reverting the traffic switch.

For index rollbacks, maintain immutable index snapshots with timestamps. New documents append to a separate temporary index during updates. If quality metrics degrade, the system can revert to the previous snapshot while preserving new documents for reprocessing. Implement automated rollback triggers based on metrics like retrieval relevance scores below threshold, latency exceeding SLA limits, or error rates spiking above baseline.

A/B Testing Approaches

RAG system A/B testing requires careful consideration of both retrieval and generation

components. Implement feature flags at multiple levels: embedding model selection, retrieval strategies, reranking algorithms, and generation prompts. Use deterministic hashing of user IDs for consistent experiment assignment across sessions.

Interleaving experiments provide more sensitive testing for retrieval quality. Rather than showing different results to different users, interleave results from both systems for the same query, tracking which results users engage with. This eliminates user population biases and provides faster convergence on statistical significance.

Shadow testing runs new configurations in parallel without affecting users. The shadow system processes real queries, with results logged but not served. Compare shadow outputs against production for quality metrics, latency analysis, and resource utilization. This identifies issues before user exposure.

Success metrics should span retrieval quality (click-through rate, dwell time), generation quality (user feedback scores, task completion rates), and system performance (latency percentiles, cost per query). Implement gradual rollout strategies: start with 1% traffic, monitor for 24 hours, then progressively increase to 5%, 25%, 50%, and finally 100% based on metric thresholds.

Multi-tenancy

SaaS software often has multiple tenants, balancing simplicity and privacy. For multi-tenancy in RAG systems, the goal is to build a system that not only finds information effectively but also respects each user's data limits. In simpler terms, every user's interaction with the system is separate, ensuring the system only looks at and uses the information that is meant for that user.

One of the simple ways to build multi-tenancy is by using metadata. When we add documents to the

system, we include specific user details in the metadata. This way each document is tied to a particular user. When someone searches, the system uses this metadata to filter and only show documents related to that user. It then does a smart search to find the most important information for that user. This approach stops private information from being mixed up between different users, keeping each person's data safe and private.

Security, Privacy and Compliance

RAG systems processing personal data must implement privacy-by-design principles. Data minimization requires indexing only necessary information, with PII stripped from embeddings where possible. Implement purpose limitation by maintaining separate indices for different use cases, preventing scope creep.

Right to erasure (Article 17) demands efficient PII removal from all system components. Maintain mapping between user identifiers and document chunks, enabling cascade deletion across embeddings, cache, and indices. Implement automated purge workflows triggered by deletion requests, with confirmation logs for compliance records.

Data portability (Article 20) requires extraction capabilities for user-specific information. Design systems to export all data associated with an individual, including documents, interaction history, and derived embeddings. Provide standardized formats (JSON, CSV) for data transfer.

Implement privacy-preserving techniques: differential privacy in embeddings to prevent membership inference attacks, federated learning approaches for sensitive domains, and homomorphic encryption for computation on encrypted embeddings where regulatory requirements demand.

Log Structure: Implement structured logging with consistent schema. Include correlation IDs for tracing requests across services. Add classification levels for sensitive operations. Ensure timestamp synchronization across distributed components.

Monitoring Strategy: Real-time alerting for suspicious patterns like unusual query volumes, access to numerous unrelated documents, or attempts to extract full dataset. Implement anomaly detection using baseline behavior analysis. Track metrics like unauthorized access attempts, PII exposure incidents, and compliance violation indicators.

Retention and Analysis: Define retention periods based on regulatory requirements (typically 6-12 months minimum). Implement log forwarding to SIEM systems for correlation. Regular automated analysis for compliance reporting. Immutable audit trail using append-only storage or blockchain for high-security requirements.

Compliance Reporting: Generate automated compliance reports showing data access patterns, privacy request fulfillment metrics, security incident summaries, and audit trail integrity verification. Maintain evidence of compliance for regulatory audits including documented procedures, technical controls verification, and regular compliance assessments.

Conclusion

It should be clear that building a robust and scalable enterprise RAG system involves carefully orchestrating interconnected components. From user authentication to input guardrails, query rewriting, encoding, document ingestion, and retrieval components, such as vector databases and generators, every step plays a crucial role in shaping the system's performance.

In the ever-evolving landscape of RAG systems, we hope this practical guide helps empower developers and leaders with actionable insights!

Galileo is the leading platform for rapid evaluation, experimentation, and observability for teams building LLM-powered applications. It is powered by a suite of metrics to identify and mitigate hallucinations.

Sign up for your free Galileo account today, or continue your Mastering RAG journey with our free, comprehensive eBook.





Pratik Bhavsar

Book a Demo

Ask AI for a summary about Galileo

LinkedIn

YouTube

Podcast

X

GitHub

Platform Overview
Agent Reliability
Signals
Luna-2
Protect
Docs
Pricing
Company
Careers
Responsible Disclosure Program

Case Studies
Blog
Mastering RAG eBook
Mastering Agents
Mastering LLM as a Judge
Mastering Multi Agent Systems
Mastering GenAI Series
Eval Engineering
Agent Leaderboard
Research
Sign up

© 2026 Galileo. All rights reserved.

Terms Privacy